

Explicación del proceso, las herramientas utilizadas, y un resumen de los inconvenientes y sus soluciones

✓ Explicación del proceso y herramientas utilizadas

📌 Objetivo del flujo

El objetivo fue diseñar un flujo automatizado con **n8n** que permita procesar facturas cargadas por los usuarios a través de un formulario de **Airtable**, extrayendo información relevante mediante inteligencia artificial, y almacenando los datos procesados en otra tabla dentro de la misma base de Airtable.

🔧 Herramientas utilizadas

◆ Airtable

- **Tabla “Subidas de Facturas”:**
Donde se cargan las facturas a través de un formulario público. Campos utilizados:
 - Archivo (adjunto)
 - Estado (Pendiente / Procesado)
- **Tabla “Facturas Procesadas”:**
Donde se almacenan los datos extraídos:
 - Número de factura
 - Fecha
 - Monto total
 - Nombre del proveedor
 - Concepto de gasto

◆ n8n (herramienta de automatización)

Se utilizaron los siguientes nodos para componer el flujo:

1. **Airtable Trigger**
Detecta cuando se crea un nuevo registro en “Subidas de Facturas”.
2. **HTTP Request**
Descarga el archivo PDF subido, en formato binario.
3. **Extract from File**
Convierte el archivo PDF a texto plano legible.

4. **AI Transform**

Se usó como nodo intermedio para generar automáticamente código en JavaScript que extraiga del texto los datos necesarios.

5. **Code**

Ejecuta el código JavaScript generado anteriormente, utilizando expresiones regulares para identificar los campos clave.

6. **Airtable (Create Record)**

Inserta los datos procesados en la tabla “Facturas Procesadas”.

Flujo del proceso paso a paso

1. El usuario sube una factura en PDF desde un formulario de Airtable.
 2. El nodo **Airtable Trigger** detecta automáticamente un nuevo registro.
 3. Se extrae la **URL del archivo adjunto**, que luego es utilizada por el nodo **HTTP Request** para descargar el archivo en binario.
 4. El nodo **Extract from File** convierte ese binario en texto plano.
 5. El nodo **AI Transform** se utilizó para generar un script de JavaScript que identifique en el texto los campos requeridos.
 6. Ese script se trasladó al nodo **Code**, donde se ejecuta directamente, devolviendo los valores estructurados.
 7. Finalmente, los datos se almacenan en la tabla “Facturas Procesadas” mediante el nodo **Airtable (Create Record)**.
 8. Como paso opcional, también puede actualizarse el estado del archivo original en “Subidas de Facturas” a “Procesado”.
-

Resumen de inconvenientes, causas y soluciones

1. **Problemas con el nodo OpenAI**

- **Causa:**

Al intentar configurar el nodo de OpenAI, se encontró que:

- Una cuenta no tenía saldo disponible.
- Otra cuenta con suscripción activa tenía un token sin los permisos suficientes para invocar el modelo.

- **Consecuencia:**

El nodo no pudo ejecutarse, y el flujo fallaba en el paso de procesamiento.

- **Solución:**

Se descartó el uso de OpenAI y se optó por una alternativa local y gratuita: **AI Transform**.

2. ❌ Fallo con alternativa Hugging Face

- **Causa:**
Se intentó usar un modelo alojado en Hugging Face mediante un nodo HTTP Request.
Al configurar el header de autorización y el body, la respuesta fue error 403.
- **Consecuencia:**
No fue posible enviar correctamente el archivo ni obtener la estructura JSON de salida.
- **Solución:**
Se descartó Hugging Face como alternativa viable y se reafirmó la decisión de utilizar **AI Transform + Code** como solución embebida en n8n, sin depender de terceros.

3. ❌ PDFs no compatibles con extracción de texto

- **Causa:**
Algunos archivos PDF, aunque visualmente legibles, fueron creados mediante sistemas como *Microsoft Print to PDF* que no almacenan el texto de forma accesible para herramientas automáticas.
- **Consecuencia:**
El nodo **Extract from File** devolvía el campo text vacío (""), haciendo imposible extraer datos.
- **Solución:**
Se reescribió el contenido de la factura en Word, y luego se guardó como PDF (Exportar > PDF) para garantizar texto real y seleccionable.
Al volver a cargar ese archivo, el flujo funcionó correctamente.

4. ❌ Código en nodo Code sin resultados

- **Causa:**
Al inicio, el nodo Code no devolvía salida porque el texto de entrada no contenía información útil (por el problema del PDF mencionado arriba).
- **Consecuencia:**
La tabla “Facturas Procesadas” quedaba vacía o con campos nulos.
- **Solución:**
Una vez corregido el PDF, se usó el nodo **AI Transform** para generar expresiones regulares, que luego se pegaron en el nodo Code.
Este nodo pudo extraer correctamente los datos y generar una salida JSON válida.

5. ❌ Campos vacíos en Airtable

- **Causa:**
Algunos campos de Airtable estaban configurados para aceptar solo valores numéricos, lo que impedía mapear expresiones dinámicas como `{{$json.totalAmount}}`.
 - **Consecuencia:**
Al intentar ejecutar el nodo “Create Record”, los campos quedaban vacíos o generaban errores.
 - **Solución:**
Se cambiaron los tipos de campo en Airtable (por ejemplo, de “número” a “texto”) para permitir el uso de valores dinámicos desde n8n.
-

6. ❌ Problemas con “Expression Editor”

- **Causa:**
En algunos campos del nodo “Create Record”, no aparecía el ícono de variable dinámica (fx) y no se podía insertar expresiones.
 - **Solución:**
Se activó manualmente el **modo Expression Editor** en cada campo afectado para ingresar los valores como `{{$json.invoiceNumber}}`, etc.
-

✅ Conclusión

A pesar de los desafíos, se logró desarrollar un flujo completamente funcional, sin depender de herramientas externas (como OpenAI o Hugging Face), aprovechando únicamente capacidades integradas de **n8n** y **Airtable**.

El uso del nodo **AI Transform** fue clave para resolver el problema del procesamiento inteligente de texto y permitió mantener el flujo 100% autónomo, replicable y escalable.